Review the unstructured csv files

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:
```
# Load the CSV files
products_df = pd.read_csv("PRODUCTS_TAKEHOME.csv")
transactions_df = pd.read_csv("TRANSACTION_TAKEHOME.csv")
users_df = pd.read_csv("USER_TAKEHOME.csv")
```

In [3]:
```
# Set up figure size for plots
plt.rcParams["figure.figsize"] = (10, 5)
```

In [4]:
```
# checking the missing value for three files
missing_values_products = products_df.isnull().sum()
missing_values_transactions = transactions_df.isnull().sum()
missing_values_users = users_df.isnull().sum()
```

In [5]:
```
data_missing_issues = {
    "Missing Values - Products": missing_values_products,
    "Missing Values - Transactions": missing_values_transactions,
    "Missing Values - Users": missing_values_users
}
print(data_missing_issues)
```

```
{'Missing Values - Products': CATEGORY_1         111
CATEGORY_2        1424
CATEGORY_3       60566
CATEGORY_4      778093
MANUFACTURER    226474
BRAND           226472
BARCODE           4025
dtype: int64, 'Missing Values - Transactions': RECEIPT_ID           0
PURCHASE_DATE        0
SCAN_DATE            0
STORE_NAME           0
USER_ID              0
BARCODE           5762
FINAL_QUANTITY       0
FINAL_SALE           0
dtype: int64, 'Missing Values - Users': ID                  0
CREATED_DATE        0
BIRTH_DATE       3675
STATE            4812
LANGUAGE        30508
GENDER           5892
dtype: int64}
```

In [6]:
```
# 2. Checking for Duplicates
duplicates_products = products_df.duplicated().sum()
duplicates_transactions = transactions_df.duplicated().sum()
duplicates_users = users_df.duplicated().sum()
```

In [7]:
```
data_duplicates_issues = {
    "Duplicate Rows - Products": duplicates_products,
    "Duplicate Rows - Transactions": duplicates_transactions,
    "Duplicate Rows - Users": duplicates_users
}
print(data_duplicates_issues)
```

```
{'Duplicate Rows - Products': 215, 'Duplicate Rows - Transactions': 17
1, 'Duplicate Rows - Users': 0}
```

In [8]:
```
# 3. Checking Unique Values in Each Column (For Better Understanding of
unique_values_products = products_df.nunique()
unique_values_transactions = transactions_df.nunique()
unique_values_users = users_df.nunique()
```

In [9]:
```
data_Unique_Values_issues = {
    "Unique Values - Products": unique_values_products,
    "Unique Values - Transactions": unique_values_transactions,
    "Unique Values - Users": unique_values_users
}
print(data_Unique_Values_issues)
```

```
{'Unique Values - Products': CATEGORY_1          27
CATEGORY_2         121
CATEGORY_3         344
CATEGORY_4         127
MANUFACTURER      4354
BRAND             8122
BARCODE         841342
dtype: int64, 'Unique Values - Transactions': RECEIPT_ID        24440
PURCHASE_DATE        89
SCAN_DATE         24440
STORE_NAME          954
USER_ID           17694
BARCODE           11027
FINAL_QUANTITY       87
FINAL_SALE         1435
dtype: int64, 'Unique Values - Users': ID              100000
CREATED_DATE     99942
BIRTH_DATE       54721
STATE               52
LANGUAGE             2
GENDER              11
dtype: int64}
```

In [10]:
```
# 4. Checking Data Types
data_types_products = products_df.dtypes
data_types_transactions = transactions_df.dtypes
data_types_users = users_df.dtypes
```

In [11]:
```
data_Types_issues = {
    "Data Types - Products": data_types_products,
    "Data Types - Transactions": data_types_transactions,
    "Data Types - Users": data_types_users
}
print(data_Types_issues)
```

```
{'Data Types - Products': CATEGORY_1       object
CATEGORY_2       object
CATEGORY_3       object
CATEGORY_4       object
MANUFACTURER     object
BRAND            object
BARCODE         float64
dtype: object, 'Data Types - Transactions': RECEIPT_ID         object
PURCHASE_DATE      object
SCAN_DATE          object
STORE_NAME         object
USER_ID            object
BARCODE           float64
FINAL_QUANTITY     object
FINAL_SALE         object
dtype: object, 'Data Types - Users': ID              object
CREATED_DATE    object
BIRTH_DATE      object
STATE           object
LANGUAGE        object
GENDER          object
dtype: object}
```

In [12]:
```
# 5. Checking for Unusual Values
# Transactions: Checking for Negative Barcodes and Strange Quantities
negative_barcodes_transactions = transactions_df[transactions_df['BARCOD
zero_or_invalid_quantities = transactions_df[transactions_df['FINAL_QUAN
```

In [13]:
```
data_Unusual_Values_issues = {
    "Negative Barcodes in Transactions": negative_barcodes_transactions.
    "Zero or Invalid Quantities": zero_or_invalid_quantities.shape[0]
}
print(data_Unusual_Values_issues)
```

```
{'Negative Barcodes in Transactions': 8, 'Zero or Invalid Quantities':
12500}
```

In [14]:
```
# 6. Checking for Placeholder or Suspicious Values
placeholder_manufacturers = products_df[products_df['MANUFACTURER'].str.
placeholder_brands = products_df[products_df['BRAND'].str.contains("PLAC
```

In [15]:
```
data_PlaceholderORSuspicious_Values_issues = {
    "Placeholder Manufacturers": placeholder_manufacturers.shape[0],
    "Placeholder Brands": placeholder_brands.shape[0]
}
print(data_PlaceholderORSuspicious_Values_issues)
```

```
{'Placeholder Manufacturers': 86902, 'Placeholder Brands': 0}
```

In [16]:
```
# 7. Checking for Unexpected Values in Categorical Columns
unexpected_genders = users_df[~users_df['GENDER'].isin(['male', 'female'
```

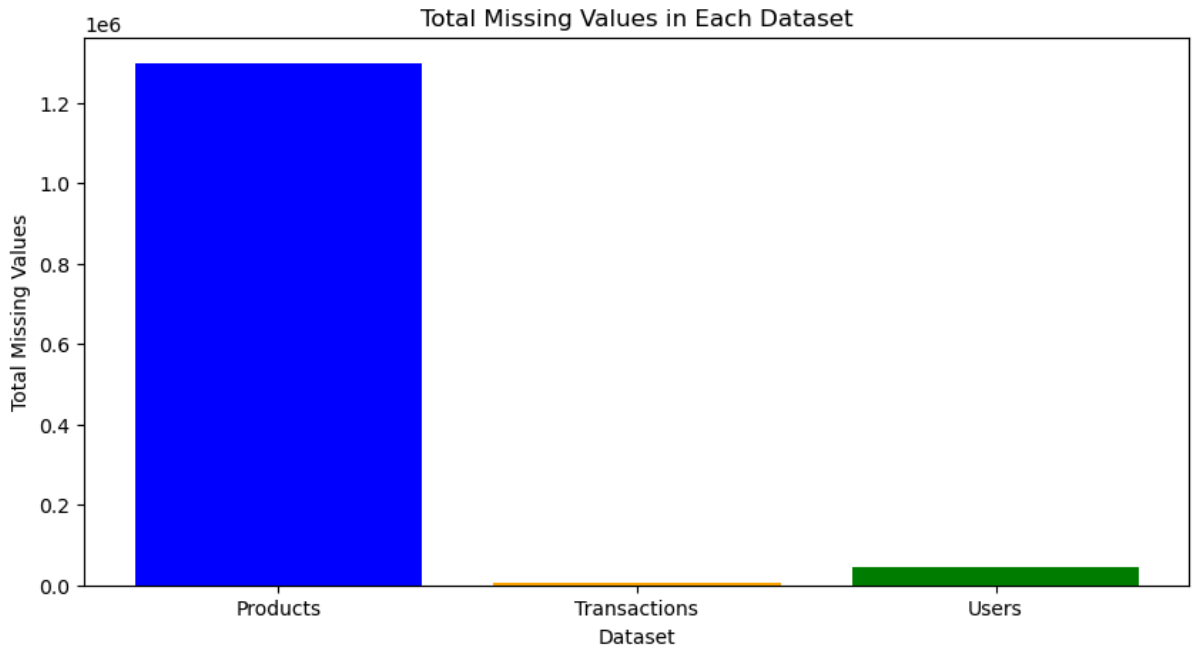
In [17]:
```
data_Unexpected_ValuesINCategorical_issues = {
        "Unexpected Gender Values": unexpected_genders.shape[0]
}
print(data_Unexpected_ValuesINCategorical_issues)
```

```
{'Unexpected Gender Values': 9931}
```

Visualization

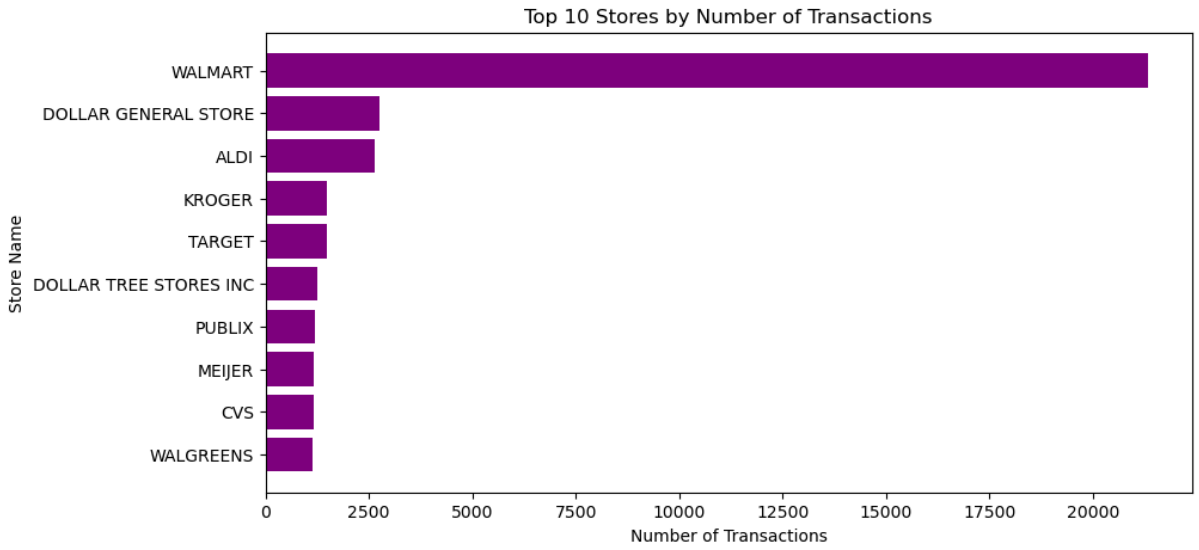
In [18]:
```
# 1.Visualization of Missing Values
missing_values = {
    "Products": products_df.isnull().sum().sum(),
    "Transactions": transactions_df.isnull().sum().sum(),
    "Users": users_df.isnull().sum().sum(),
}

plt.bar(missing_values.keys(), missing_values.values(), color=["blue", "
plt.xlabel("Dataset")
plt.ylabel("Total Missing Values")
plt.title("Total Missing Values in Each Dataset")
plt.show()
```



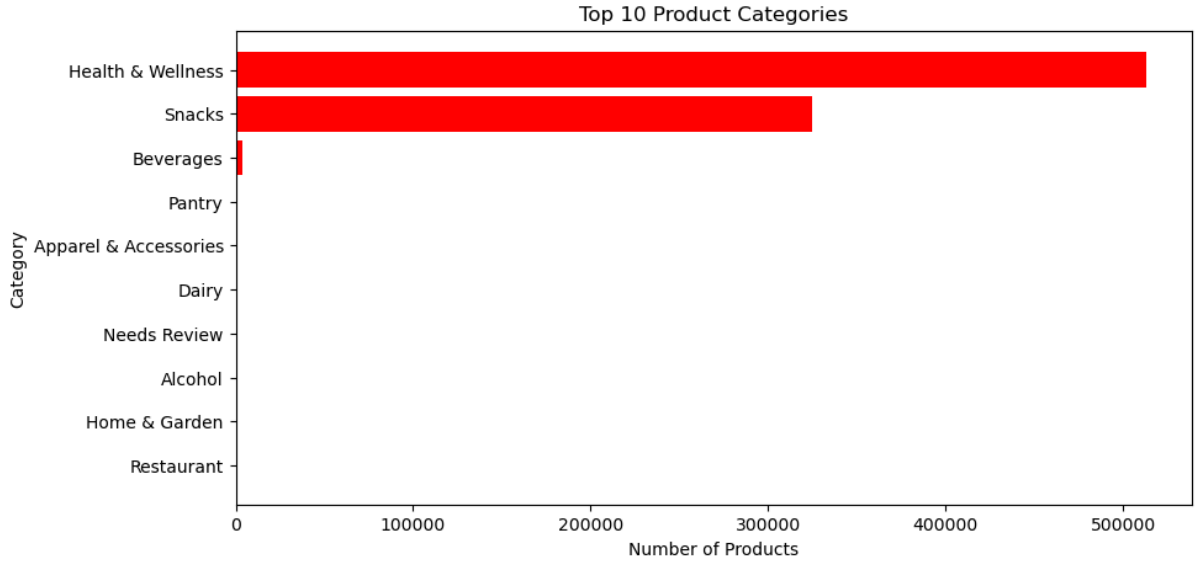
In [19]:
```
# 2. Distribution of Transactions per Store
store_counts = transactions_df["STORE_NAME"].value_counts().head(10)  #

plt.barh(store_counts.index, store_counts.values, color="purple")
plt.xlabel("Number of Transactions")
plt.ylabel("Store Name")
plt.title("Top 10 Stores by Number of Transactions")
plt.gca().invert_yaxis()
plt.show()
```



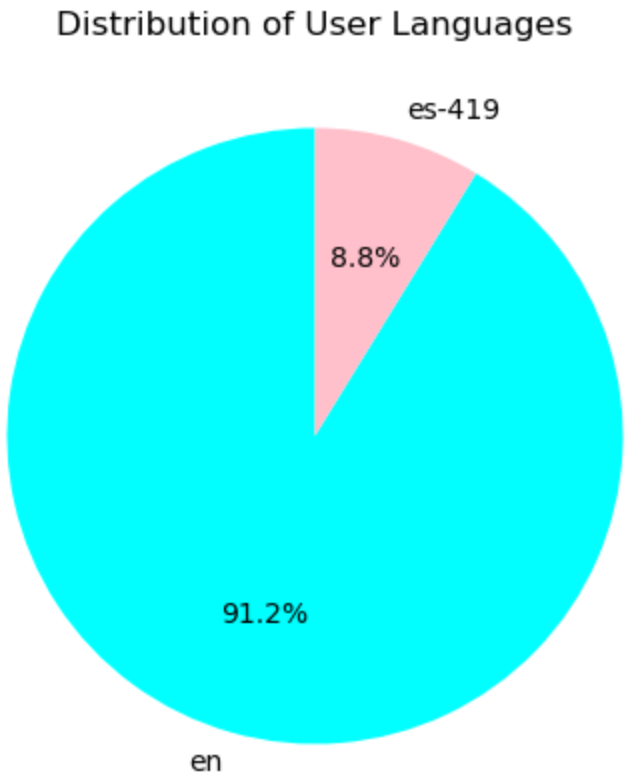
In [20]:
```
# 3. Distribution of Product Categories
category_counts = products_df["CATEGORY_1"].value_counts().head(10)  # T

plt.barh(category_counts.index, category_counts.values, color="red")
plt.xlabel("Number of Products")
plt.ylabel("Category")
plt.title("Top 10 Product Categories")
plt.gca().invert_yaxis()
plt.show()
```



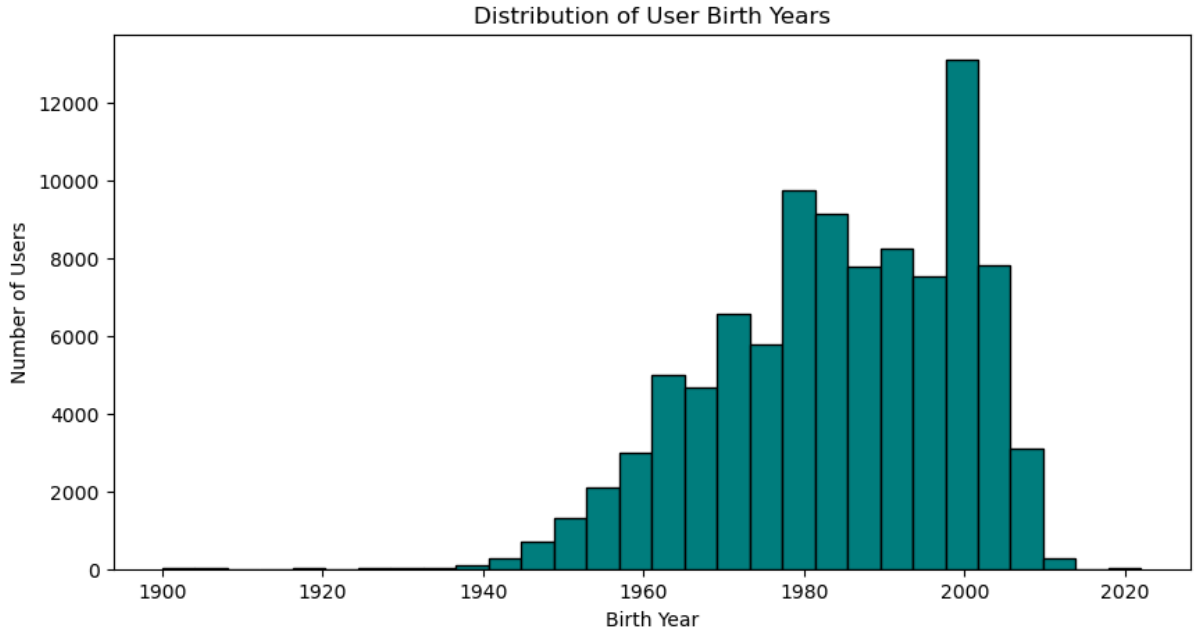
In [21]:
```
# 4. Distribution of User Languages
language_counts = users_df["LANGUAGE"].value_counts()

plt.pie(language_counts, labels=language_counts.index, autopct="%1.1f%%"
plt.title("Distribution of User Languages")
plt.show()
```



In [22]:
```
# 5. Age Distribution of Users (Based on Birth Year)
users_df["BIRTH_YEAR"] = pd.to_datetime(users_df["BIRTH_DATE"], errors="
users_df.dropna(subset=["BIRTH_YEAR"], inplace=True)

plt.hist(users_df["BIRTH_YEAR"], bins=30, color="teal", edgecolor="black
plt.xlabel("Birth Year")
plt.ylabel("Number of Users")
plt.title("Distribution of User Birth Years")
plt.show()
```



In [ ]: